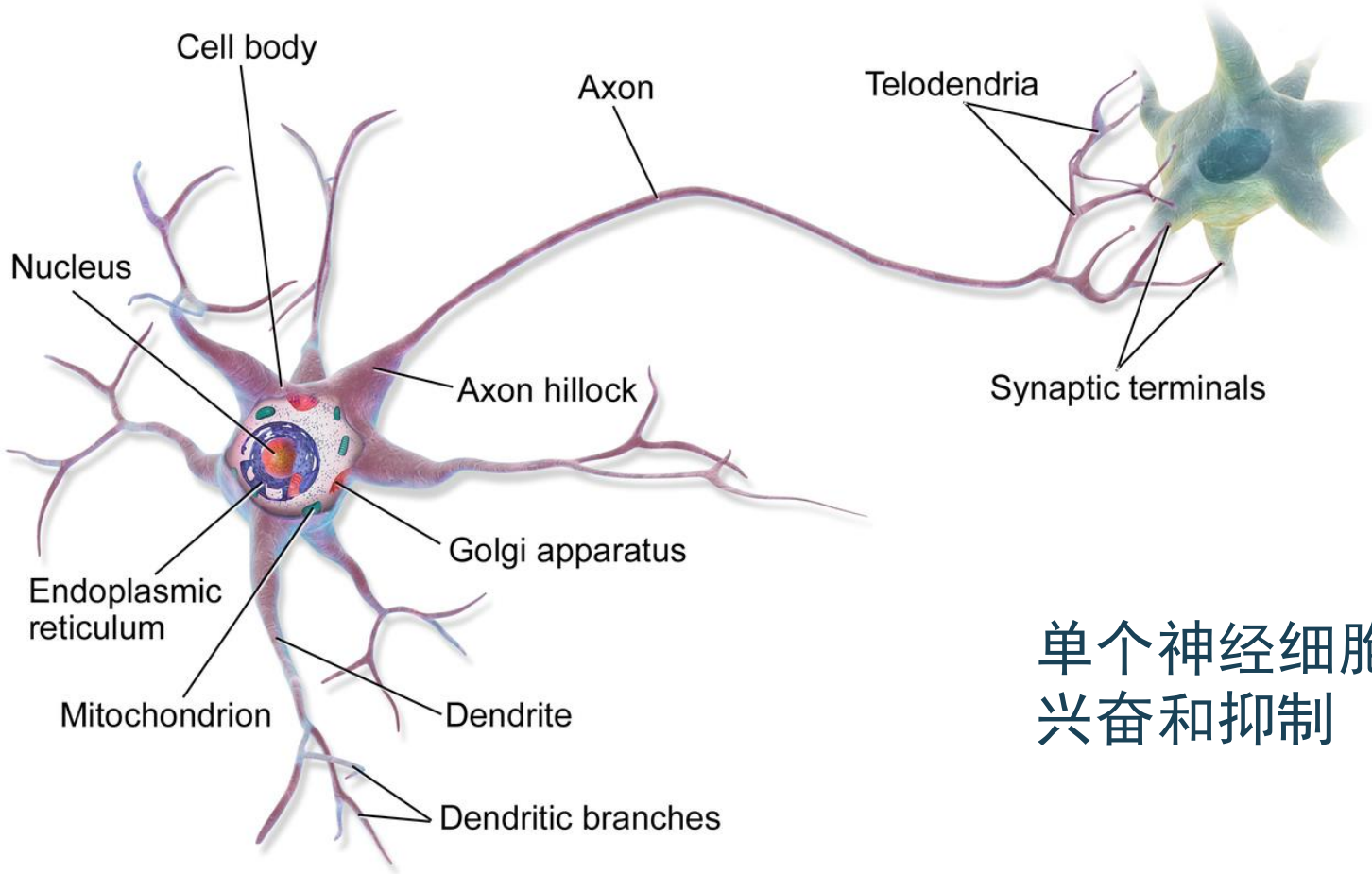
前馈神经网络

《神经网络与深度学习》第4讲

提纲

- 神经网络概览
- 前馈神经网络
- 激活函数
- 反向传播算法
- 优化问题

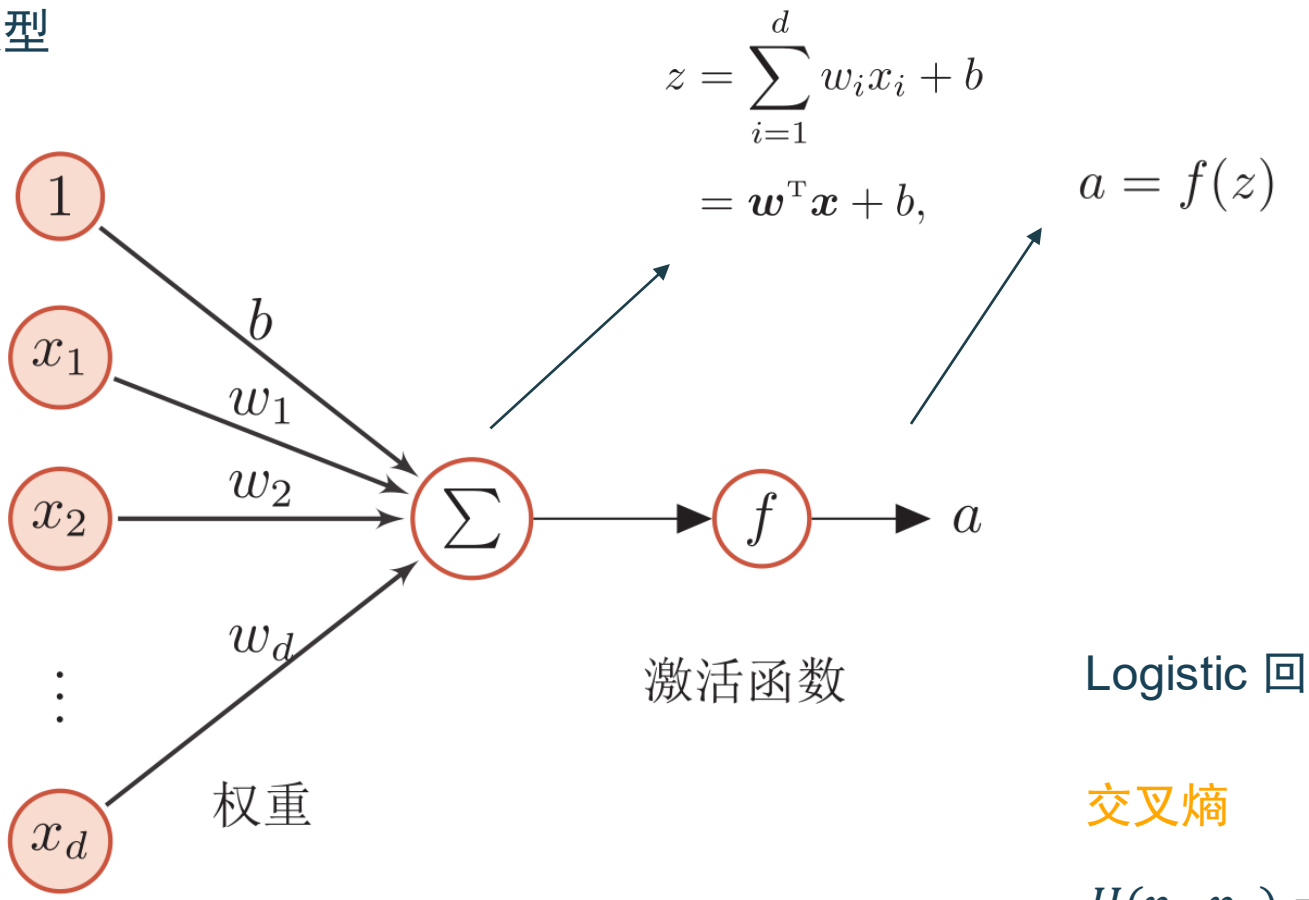
生物神经元



单个神经细胞只有两种状态：
兴奋和抑制

人工神经元

一个简单的线性模型



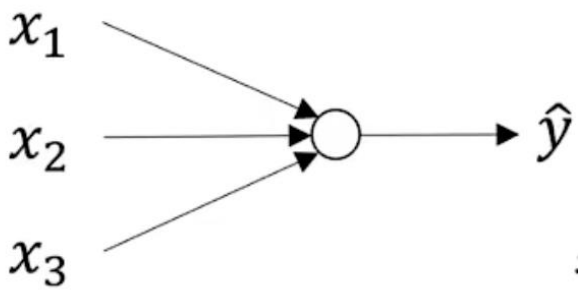
Logistic 回归 $f(z) = \dfrac{1}{1+\exp(-z)}$

交叉熵

$$H(p_r, p_\theta) = -(y^* \log \hat{y} + (1-y^*)\log(1-\hat{y}))$$

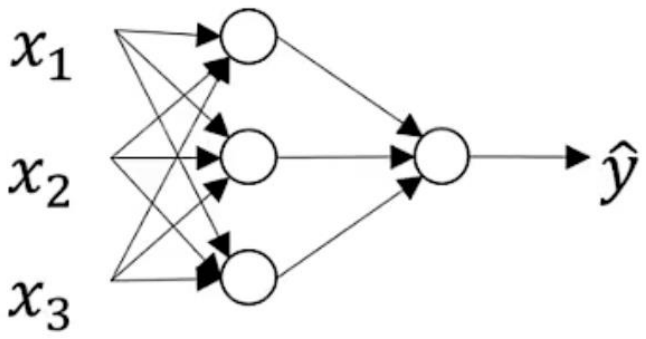
梯度下降法

神经网络的例子



$$\boxed{z = \boldsymbol{w}\boldsymbol{x} + b \longrightarrow a = \sigma(z)} \longrightarrow \hat{y} = a$$

两层神经网络



$$\boxed{\boldsymbol{z}^{(1)} = \boldsymbol{w}^{(1)}\boldsymbol{x} + b^{(1)} \longrightarrow \boldsymbol{a}^{(1)} = \sigma(\boldsymbol{z}^{(1)})} \longrightarrow \boxed{z^{(2)} = \boldsymbol{w}^{(2)}\boldsymbol{a}^{(1)} + \boldsymbol{b}^{(2)} \longrightarrow a^{(2)} = \sigma(z^{(2)})} \longrightarrow \hat{y} = a^{(2)}$$

人工神经网络

人工神经网络主要由大量的神经元以及它们之间的有向连接构成

神经元的激活规则

主要是指神经元输入到输出之间的映射关系，一般为非线性函数

网络的拓扑结构

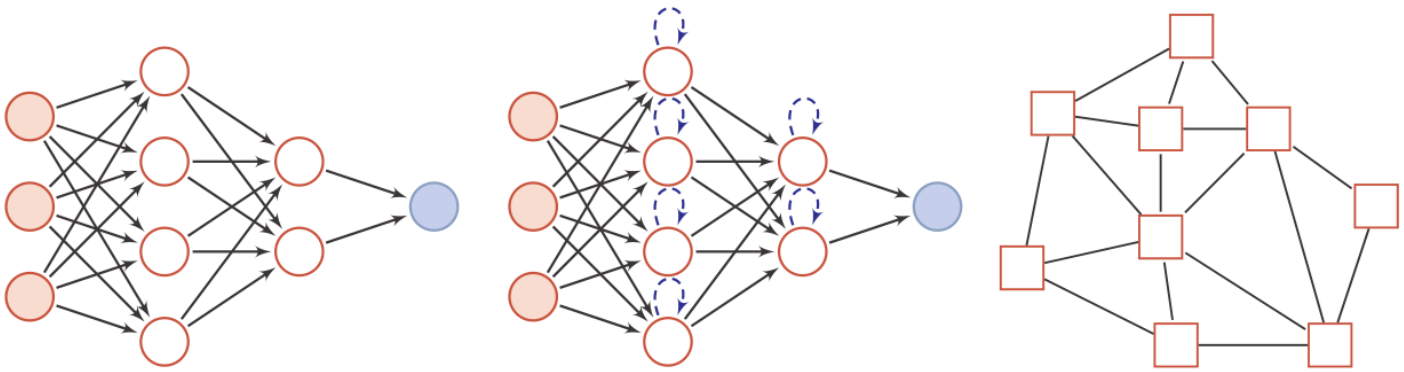
不同神经元之间的连接关系

学习算法

通过训练数据来学习神经网络的参数

网络结构

人工神经网络由神经元模型构成，这种由许多神经元组成的信息处理网络具有并行分布结构.



(a) 前馈网络　(b) 记忆网络　(c) 图网络

圆形节点表示一个神经元，方形节点表示一组神经元。

网络结构

神经网络最早是作为一种主要的连接主义模型

20世纪80年代后期，最流行的一种连接主义模型是分布式并行处理网络

- 信息表示是分布式的（非局部的）；
- 记忆和知识是存储在单元之间的连接上；
- 通过逐渐改变单元之间的连接强度来学习新的知识。

引入误差反向传播来改进其学习能力之后，神经网络也越来越多地应用在各种机器学习任务上

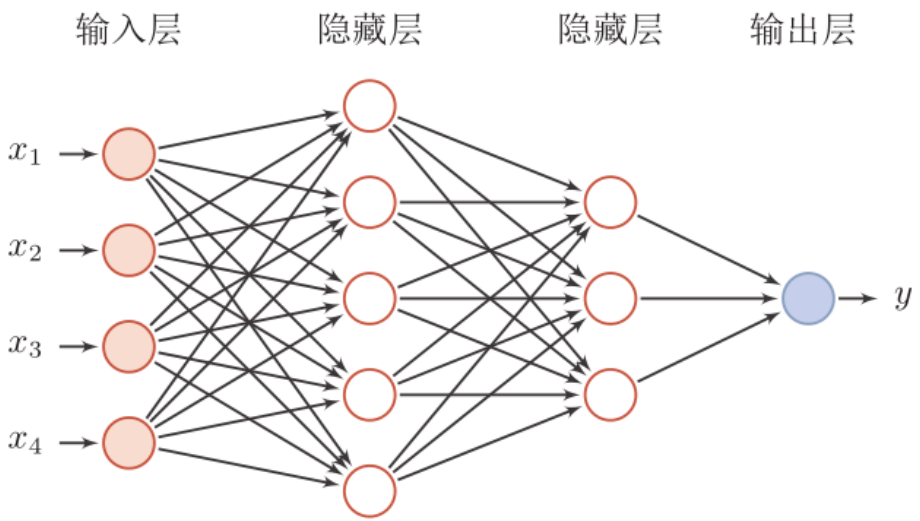
网络结构

前馈神经网络（全连接神经网络、多层感知器）

各神经元分别属于不同的层，层内无连接

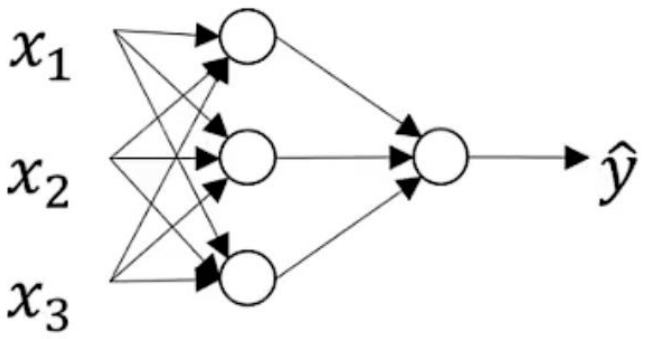
相邻两层之间的神经元全部两两连接

整个网络中无反馈，信号从输入层向输出层单向传播，可用一个有向无环图表示



神经网络的例子

2层神经网络



$$\boldsymbol{a}^{(0)} = \boldsymbol{x}$$

$$\boldsymbol{z}^{(1)} = \boldsymbol{w}^{(1)}\boldsymbol{a}^{(0)} + b^{(1)}$$ 净活性值 $$\boldsymbol{z}^{(l)} = \boldsymbol{W}^{(l)}\boldsymbol{a}^{(l-1)} + \boldsymbol{b}^{(l)},$$

$$\boldsymbol{a}^{(1)} = f(\boldsymbol{z}^{(1)})$$ 活性值 $$\boldsymbol{a}^{(l)} = f_l(\boldsymbol{z}^{(l)}).$$

$$z^{(2)} = \boldsymbol{w}^{(2)}\boldsymbol{a}^{(1)} + \boldsymbol{b}^{(2)}$$

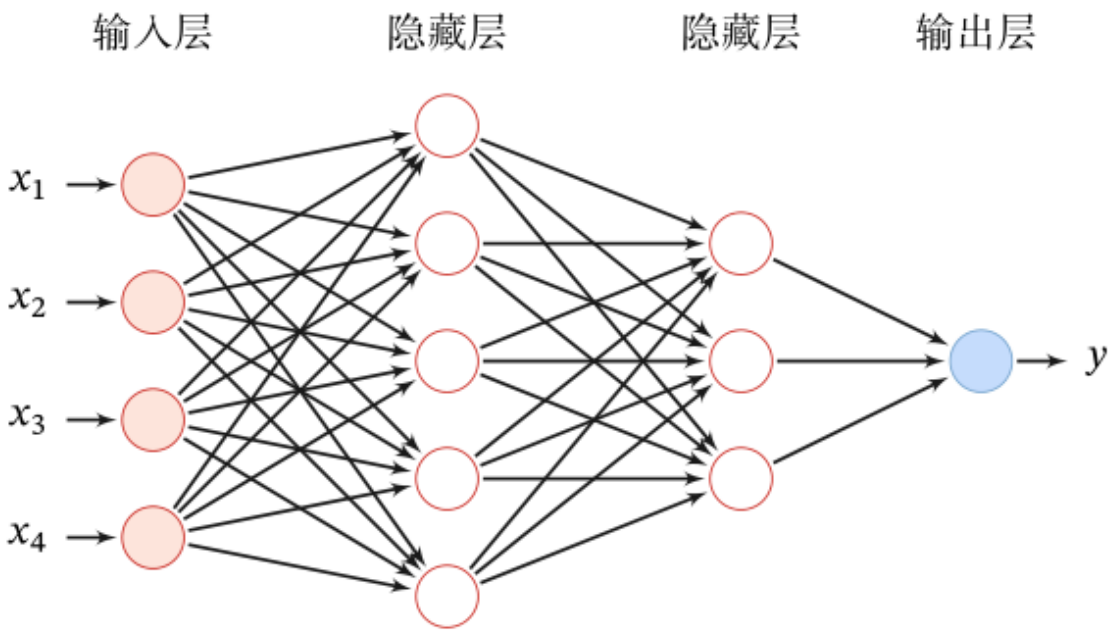
$$a^{(2)} = f(z^{(2)})$$

$$\hat{y} = a^{(2)}$$

损失函数 $\mathcal{L}(y, \hat{y})$ 如何计算损失函数对参数的偏导数?

前馈网络

给定一个前馈神经网络，用下面的记号来描述这样网络



记号	含义
L	神经网络的层数
M_l	第 l 层神经元的个数
$f_l(\cdot)$	第 l 层神经元的激活函数
$\boldsymbol{W}^{(l)} \in \mathbb{R}^{M_l \times M_{l-1}}$	第 $l-1$ 层到第 l 层的权重矩阵
$\boldsymbol{b}^{(l)} \in \mathbb{R}^{M_l}$	第 $l-1$ 层到第 l 层的偏置
$\boldsymbol{z}^{(l)} \in \mathbb{R}^{M_l}$	第 l 层神经元的净输入（净活性值）
$\boldsymbol{a}^{(l)} \in \mathbb{R}^{M_l}$	第 l 层神经元的输出（活性值）

信息传递过程

前馈神经网络通过下面公式进行信息传播

$$\boldsymbol{z}^{(l)} = \boldsymbol{W}^{(l)}\boldsymbol{a}^{(l-1)} + \boldsymbol{b}^{(l)}, \longrightarrow \boldsymbol{z}^{(l)} = \boldsymbol{W}^{(l)} f_{l-1}(\boldsymbol{z}^{(l-1)}) + \boldsymbol{b}^{(l)},$$

$$\boldsymbol{a}^{(l)} = f_l(\boldsymbol{z}^{(l)}). \longrightarrow \boldsymbol{a}^{(l)} = f_l(\boldsymbol{W}^{(l)}\boldsymbol{a}^{(l-1)} + \boldsymbol{b}^{(l)}).$$

前馈计算

$$\boldsymbol{x} = \boldsymbol{a}^{(0)} \to \boldsymbol{z}^{(1)} \to \boldsymbol{a}^{(1)} \to \boldsymbol{z}^{(2)} \to \cdots \to \boldsymbol{a}^{(L-1)} \to \boldsymbol{z}^{(L)} \to \boldsymbol{a}^{(L)} = \phi(\boldsymbol{x}; \boldsymbol{W}, \boldsymbol{b}))$$

通用近似定理

定理 4.1 – 通用近似定理（Universal Approximation Theorem）[Cybenko, 1989, Hornik et al., 1989]： 令 $\varphi(\cdot)$ 是一个非常数、有界、单调递增的连续函数，$\mathcal{I}_d$ 是一个 d 维的单位超立方体 $[0,1]^d$，$C(\mathcal{I}_d)$ 是定义在 $\mathcal{I}_d$ 上的连续函数集合。对于任何一个函数 $f \in C(\mathcal{I}_d)$，存在一个整数 m，和一组实数 $v_i, b_i \in \mathbb{R}$ 以及实数向量 $\mathbf{w}_i \in \mathbb{R}^d$，$i = 1, \cdots, m$，以至于我们可以定义函数

$$F(\mathbf{x}) = \sum_{i=1}^{m} v_i \varphi(\mathbf{w}_i^{\mathrm{T}} \mathbf{x} + b_i), \tag{4.33}$$

作为函数 f 的近似实现，即

$$|F(\mathbf{x}) - f(\mathbf{x})| < \epsilon, \forall \mathbf{x} \in \mathcal{I}_d. \tag{4.34}$$

其中 $\epsilon > 0$ 是一个很小的正数。

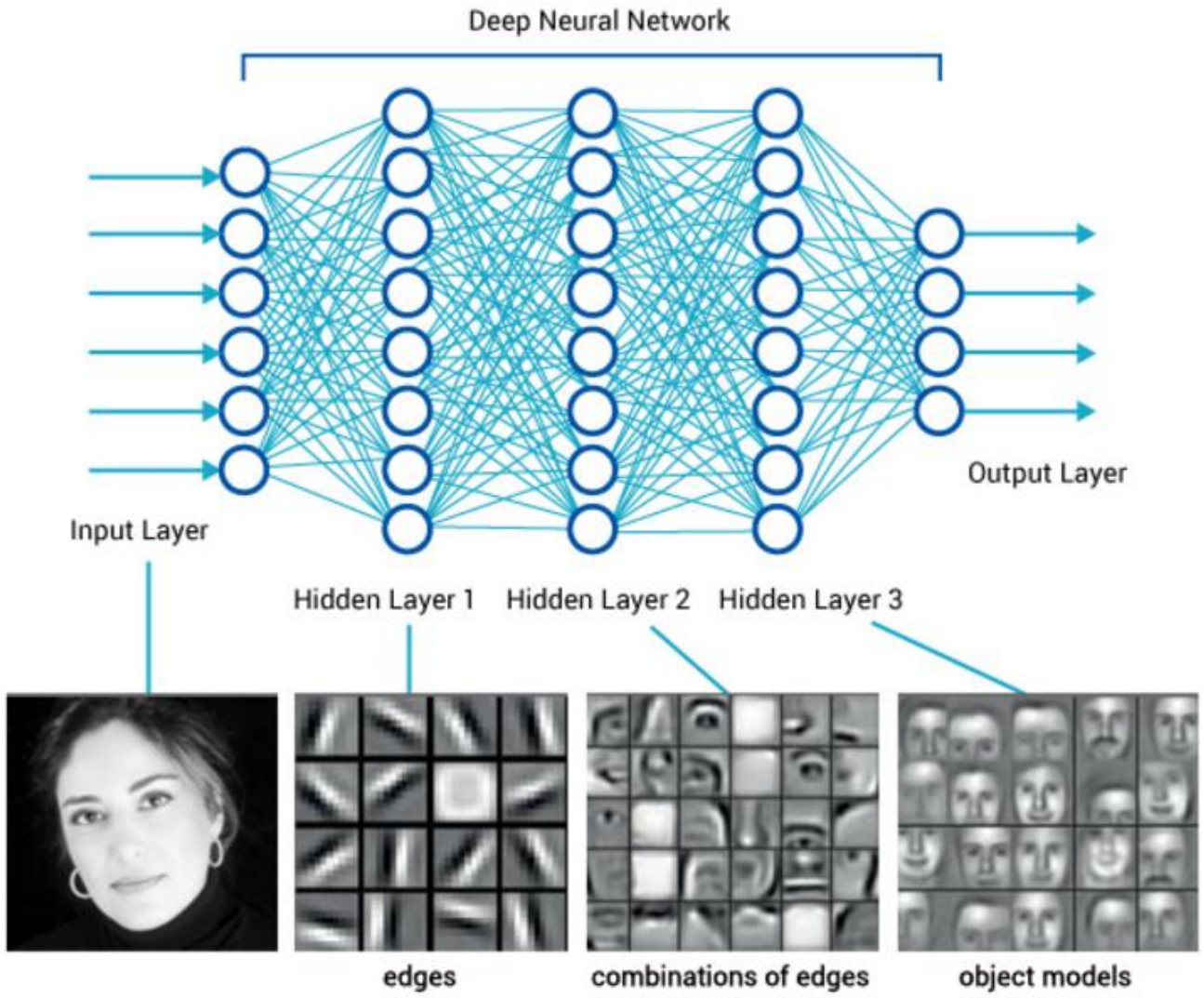
应用到机器学习

神经网络可以作为一个“万能”函数来使用，可以用来进行复杂的特征转换，或逼近一个复杂的条件分布

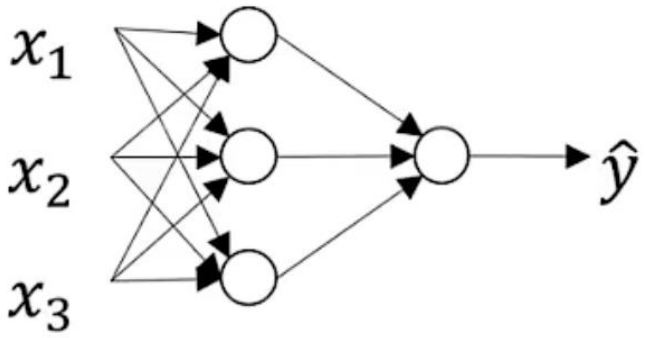
$$\hat{y} = g(\underbrace{\varphi(\mathbf{x})}_{\text{神经网络}}, \underbrace{\theta}_{\text{分类器}})$$

如果$g(\cdot)$为Logistic回归，那么Logistic回归分类器可以看成神经网络的最后一层

深层前馈神经网络


Deep Neural Network
Input Layer
Hidden Layer 1
Hidden Layer 2
Hidden Layer 3
Output Layer
edges
combinations of edges
object models

两层神经网络



$$\boxed{\boldsymbol{z}^{(1)} = \boldsymbol{w}^{(1)}\boldsymbol{x} + b^{(1)} \longrightarrow \boldsymbol{a}^{(1)} = \sigma(\boldsymbol{z}^{(1)})} \longrightarrow \boxed{z^{(2)} = \boldsymbol{w}^{(2)}\boldsymbol{a}^{(1)} + \boldsymbol{b}^{(2)} \longrightarrow a^{(2)} = \sigma(z^{(2)})} \longrightarrow \hat{y} = a^{(2)}$$

激活函数怎么选?

激活函数的性质

连续并可导（允许少数点上不可导）的非线性函数

可导的激活函数可以直接利用数值优化的方法来学习网络参数

激活函数及其导函数要尽可能的简单

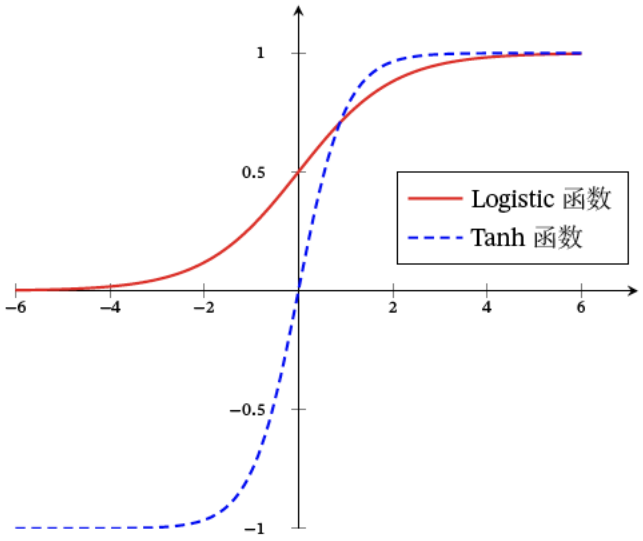
有利于提高网络计算效率

激活函数的导函数的值域要在一个合适的区间内

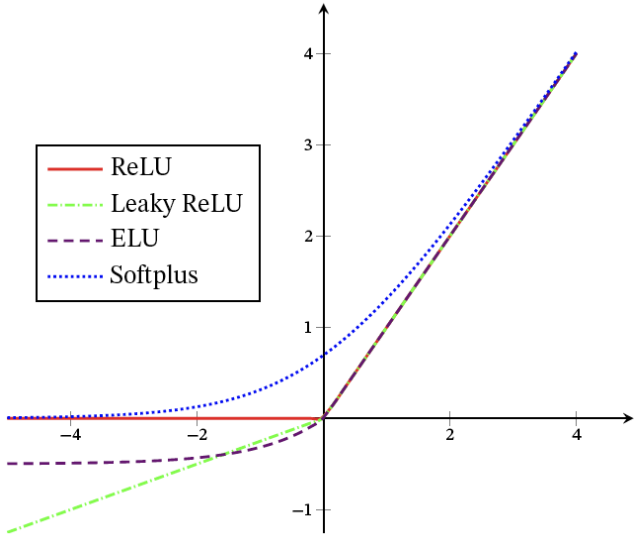
不能太大也不能太小，否则会影响训练的效率和稳定性

单调递增?

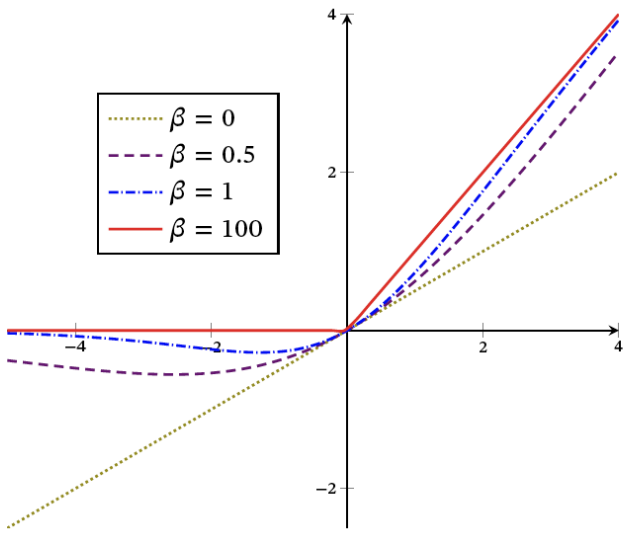
常用的激活函数



S型函数
Sigmoid Function



斜坡函数
Ramp Function



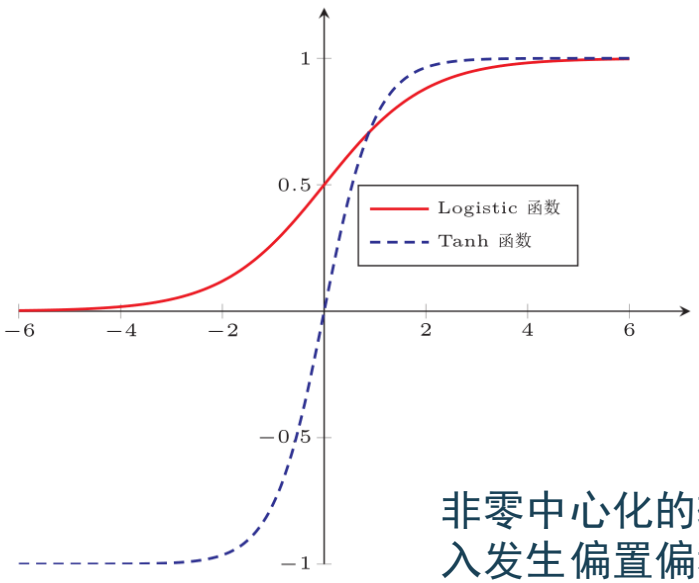
复合函数
Compound Function

S型函数

$$\sigma(x) = \frac{1}{1+\exp(-x)}$$

$$\tanh(x) = \frac{\exp(x)-\exp(-x)}{\exp(x)+\exp(-x)}$$

$$\tanh(x) = 2\sigma(2x) - 1.$$



非零中心化的输出会使得其后一层的神经元的输入发生偏置偏移（bias shift），并进一步使得梯度下降的收敛速度变慢。

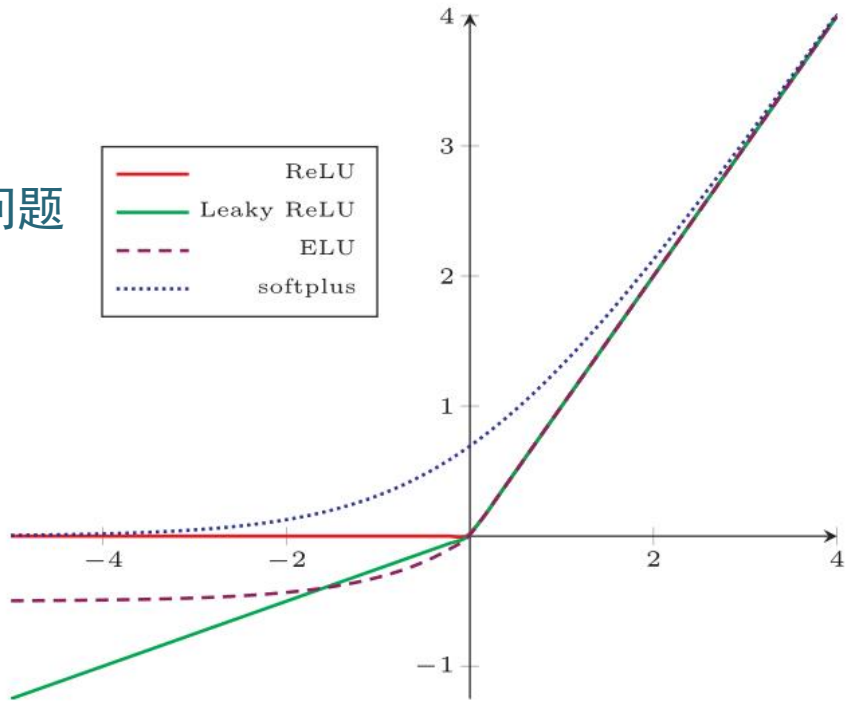
饱和函数

Tanh函数是零中心化的，而logistic函数的输出恒大于0

斜坡函数

$$\mathrm{ReLU}(x) = \begin{cases} x & x \geq 0 \\ 0 & x < 0 \end{cases}$$
$$= \max(0, x).$$

计算上更加高效
生物学合理性
　单侧抑制、宽兴奋边界
在一定程度上缓解梯度消失问题



死亡 ReLU 问题

$$\mathrm{LeakyReLU}(x) = \begin{cases} x & \text{if } x > 0 \\ \gamma x & \text{if } x \leq 0 \end{cases}$$
$$= \max(0, x) + \gamma \min(0, x)$$

近似的零中心化的非线性函数

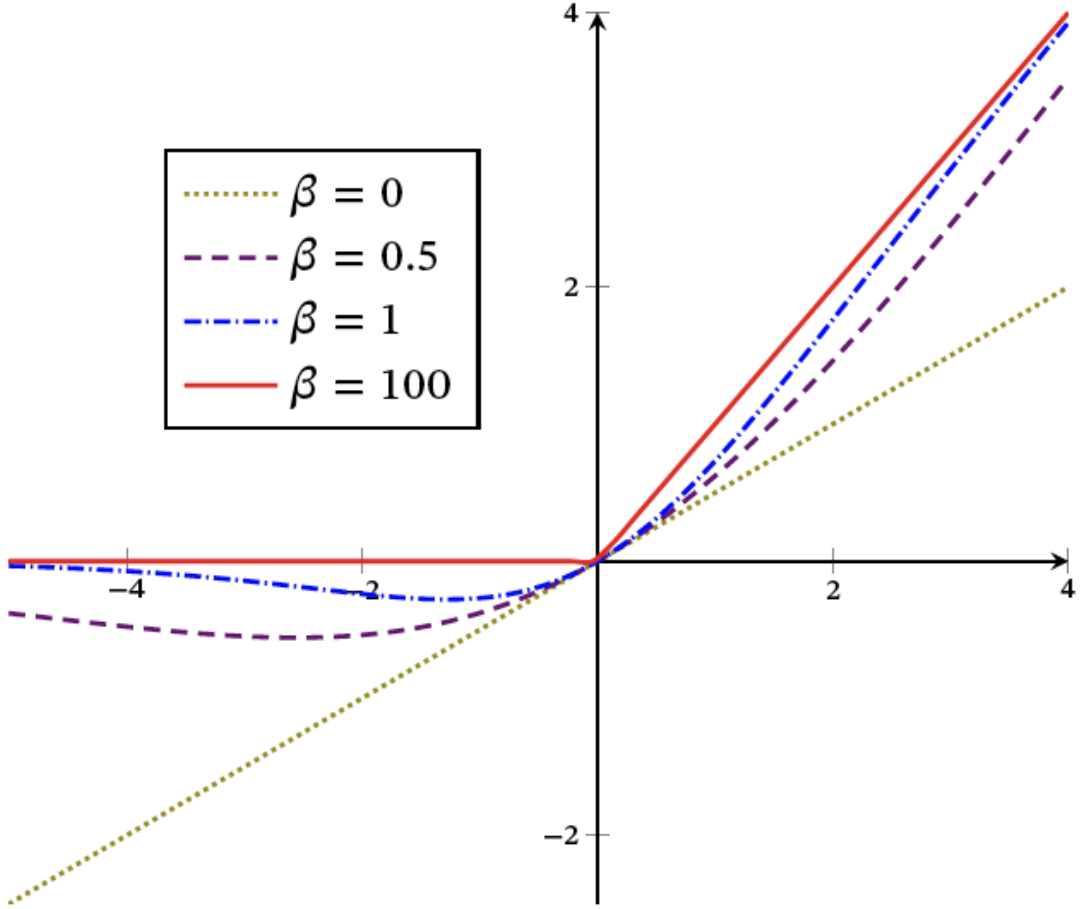
$$\mathrm{ELU}(x) = \begin{cases} x & \text{if } x > 0 \\ \gamma(\exp(x) - 1) & \text{if } x \leq 0 \end{cases}$$
$$= \max(0, x) + \min(0, \gamma(\exp(x) - 1)),$$

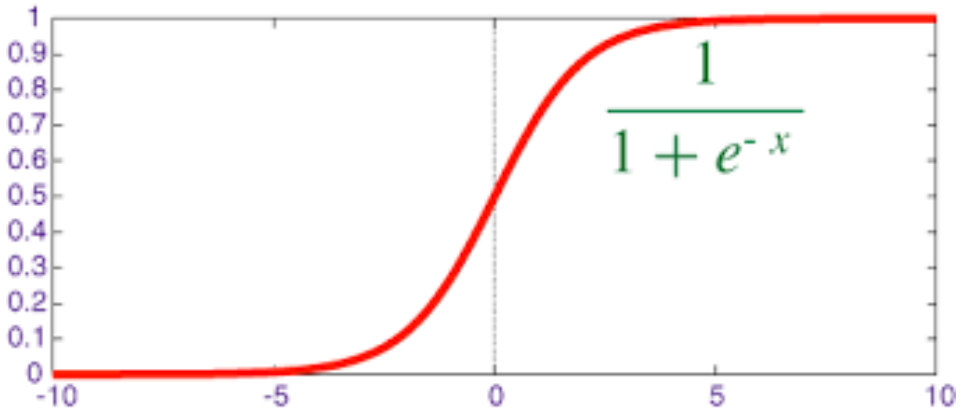
平滑版本

$$\mathrm{Softplus}(x) = \log(1 + \exp(x)).$$

复合函数

Swish函数 $$\mathrm{swish}(x) = x\sigma(\beta x)$$





复合函数

高斯误差线性单元（Gaussian Error Linear Unit，GELU）

$$\mathrm{GELU}(x) = xP(X \leq x)$$

其中$P(X \leq x)$ 是高斯分布$N(\mu, \sigma^2)$ 的分布函数，其中 μ, σ为超参数，一般设 $\mu = 0, \sigma = 1$

由于高斯分布的分布函数为S型函数，因此GELU可以用Tanh函数或Logistic函数来近似

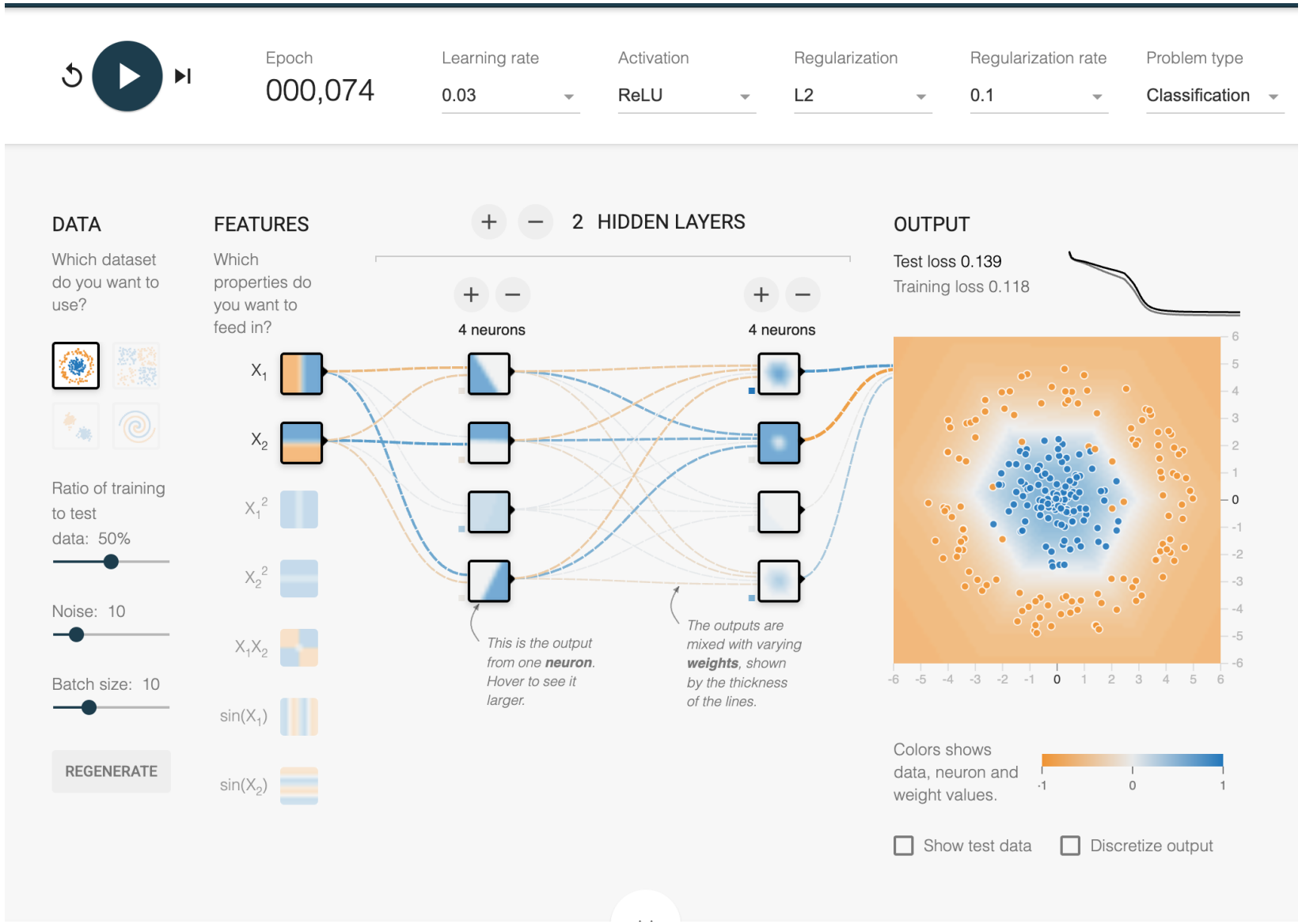
$$\mathrm{GELU}(x) \approx 0.5x\left(1 + \tanh\left(\sqrt{\frac{2}{\pi}}(x + 0.044715x^3)\right)\right)$$

或 $$\mathrm{GELU}(x) \approx x\sigma(1.702x).$$

常见激活函数及其导数

激活函数	函数	导数
Logistic 函数	$f(x) = \frac{1}{1+\exp(-x)}$	$f'(x) = f(x)\big(1 - f(x)\big)$
Tanh 函数	$f(x) = \frac{\exp(x)-\exp(-x)}{\exp(x)+\exp(-x)}$	$f'(x) = 1 - f(x)^2$
ReLU 函数	$f(x) = \max(0, x)$	$f'(x) = I(x > 0)$
ELU 函数	$f(x) = \max(0, x) + \min\Big(0, \gamma\big(\exp(x) - 1\big)\Big)$	$f'(x) = I(x > 0) + I(x \leq 0) \cdot \gamma \exp(x)$
SoftPlus 函数	$f(x) = \log\big(1 + \exp(x)\big)$	$f'(x) = \frac{1}{1+\exp(-x)}$

Playground



playground

矩阵微积分

标量关于向量的偏导数 $\boldsymbol{x} \in \mathbb{R}^M$ $\frac{\partial y}{\partial \boldsymbol{x}} = [\frac{\partial y}{\partial x_1}, \cdots, \frac{\partial y}{\partial x_M}]^\top$

向量关于标量的偏导数 $\boldsymbol{y} = f(\boldsymbol{x}) \in \mathbb{R}^N$ $\frac{\partial \boldsymbol{y}}{\partial x} = [\frac{\partial y_1}{\partial x}, \cdots, \frac{\partial y_N}{\partial x}]$ $\in \mathbb{R}^{1\times N}$

向量关于向量的偏导数

$$\frac{\partial f(\boldsymbol{x})}{\partial \boldsymbol{x}} = \begin{bmatrix} \frac{\partial y_1}{\partial x_1} & \cdots & \frac{\partial y_N}{\partial x_1} \\ \vdots & \ddots & \vdots \\ \frac{\partial y_1}{\partial x_M} & \cdots & \frac{\partial y_N}{\partial x_M} \end{bmatrix} \in \mathbb{R}^{M\times N}$$

矩阵微积分

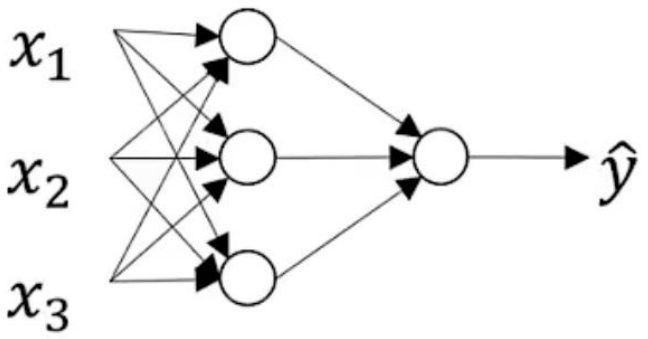
向量函数及其导数

$$\frac{\partial \boldsymbol{x}}{\partial \boldsymbol{x}} = \boldsymbol{I}, \qquad \frac{\partial \boldsymbol{A}\boldsymbol{x}}{\partial \boldsymbol{x}} = \boldsymbol{A}^{\top}, \qquad \frac{\partial \boldsymbol{x}^{\top}\boldsymbol{A}}{\partial \boldsymbol{x}} = \boldsymbol{A}.$$

链式法则 $\boldsymbol{x} \in \mathbb{R}^{M}, \boldsymbol{y} = g(\boldsymbol{x}) \in \mathbb{R}^{K}, \boldsymbol{z} = f(\boldsymbol{y}) \in \mathbb{R}^{N}$

$$\frac{\partial \boldsymbol{z}}{\partial \boldsymbol{x}} = \frac{\partial \boldsymbol{y}}{\partial \boldsymbol{x}} \frac{\partial \boldsymbol{z}}{\partial \boldsymbol{y}} \quad \in \mathbb{R}^{M \times N}.$$

神经网络



$$\boldsymbol{z}^{(l)} = \boldsymbol{W}^{(l)}\boldsymbol{a}^{(l-1)} + \boldsymbol{b}^{(l)},$$
$$\boldsymbol{a}^{(l)} = f_l(\boldsymbol{z}^{(l)}).$$

损失函数 $\mathcal{L}(y, \hat{y})$　如何计算损失函数对参数的偏导数?

$$\frac{\partial \mathcal{L}(y, \hat{y})}{\partial \boldsymbol{W}^{(l)}} \qquad \frac{\partial \mathcal{L}(y, \hat{y})}{\partial \boldsymbol{b}^{(l)}}$$

反向传播算法

$$\boldsymbol{z}^{(l)} = \boldsymbol{W}^{(l)}\boldsymbol{a}^{(l-1)} + \boldsymbol{b}^{(l)},$$
$$\boldsymbol{a}^{(l)} = f_l(\boldsymbol{z}^{(l)}).$$

$$\delta^{(l)} \triangleq \frac{\partial \mathcal{L}(\boldsymbol{y}, \hat{\boldsymbol{y}})}{\partial \boldsymbol{z}^{(l)}}$$

误差项

$$\frac{\partial \mathcal{L}(\boldsymbol{y}, \hat{\boldsymbol{y}})}{\partial w_{ij}^{(l)}} = \frac{\partial \boldsymbol{z}^{(l)}}{\partial w_{ij}^{(l)}} \frac{\partial \mathcal{L}(\boldsymbol{y}, \hat{\boldsymbol{y}})}{\partial \boldsymbol{z}^{(l)}},$$
$$\frac{\partial \mathcal{L}(\boldsymbol{y}, \hat{\boldsymbol{y}})}{\partial \boldsymbol{b}^{(l)}} = \frac{\partial \boldsymbol{z}^{(l)}}{\partial \boldsymbol{b}^{(l)}} \frac{\partial \mathcal{L}(\boldsymbol{y}, \hat{\boldsymbol{y}})}{\partial \boldsymbol{z}^{(l)}}.$$

$$\begin{aligned}
\frac{\partial \boldsymbol{z}^{(l)}}{\partial w_{ij}^{(l)}} &= \left[\frac{\partial z_1^{(l)}}{\partial w_{ij}^{(l)}}, \cdots, \boxed{\frac{\partial z_i^{(l)}}{\partial w_{ij}^{(l)}}}, \cdots, \frac{\partial z_{M_l}^{(l)}}{\partial w_{ij}^{(l)}} \right] \\
&= \left[0, \cdots, \boxed{\frac{\partial (\boldsymbol{w}_{i:}^{(l)} \boldsymbol{a}^{(l-1)} + b_i^{(l)})}{\partial w_{ij}^{(l)}}}, \cdots, 0 \right] \\
&= \left[0, \cdots, a_j^{(l-1)}, \cdots, 0 \right]
\end{aligned}$$

$$\frac{\partial \mathcal{L}(\boldsymbol{y}, \hat{\boldsymbol{y}})}{\partial \boldsymbol{W}^{(l)}} = \delta^{(l)} (\boldsymbol{a}^{(l-1)})^{\mathsf{T}} \quad \in \mathbb{R}^{M_l \times M_{l-1}}.$$

$$\frac{\partial \boldsymbol{z}^{(l)}}{\partial \boldsymbol{b}^{(l)}} = \boldsymbol{I}_{M_l} \qquad \frac{\partial \mathcal{L}(\boldsymbol{y}, \hat{\boldsymbol{y}})}{\partial \boldsymbol{b}^{(l)}} = \delta^{(l)} \quad \in \mathbb{R}^{M_l}.$$

反向传播算法

$$\boldsymbol{z}^{(l)} = \boldsymbol{W}^{(l)}\boldsymbol{a}^{(l-1)} + \boldsymbol{b}^{(l)},$$

$$\boldsymbol{a}^{(l)} = f_l(\boldsymbol{z}^{(l)}).$$

$$\delta^{(l)} \triangleq \frac{\partial \mathcal{L}(\boldsymbol{y}, \hat{\boldsymbol{y}})}{\partial \boldsymbol{z}^{(l)}}$$

$$\begin{aligned}
\delta^{(l)} &\triangleq \frac{\partial \mathcal{L}(\boldsymbol{y}, \hat{\boldsymbol{y}})}{\partial \boldsymbol{z}^{(l)}} \\
&= \boxed{\frac{\partial \boldsymbol{a}^{(l)}}{\partial \boldsymbol{z}^{(l)}}} \cdot \boxed{\frac{\partial \boldsymbol{z}^{(l+1)}}{\partial \boldsymbol{a}^{(l)}}} \cdot \boxed{\frac{\partial \mathcal{L}(\boldsymbol{y}, \hat{\boldsymbol{y}})}{\partial \boldsymbol{z}^{(l+1)}}} \\
&= \boxed{\operatorname{diag}(f_l'(\boldsymbol{z}^{(l)}))} \cdot \boxed{(\boldsymbol{W}^{(l+1)})^{\mathsf{T}}} \cdot \boxed{\delta^{(l+1)}} \\
&= f_l'(\boldsymbol{z}^{(l)}) \odot \left((\boldsymbol{W}^{(l+1)})^{\mathsf{T}}\delta^{(l+1)}\right) \quad \in \mathbb{R}^{M_l},
\end{aligned}$$

$$\frac{\partial \mathcal{L}(\boldsymbol{y}, \hat{\boldsymbol{y}})}{\partial \boldsymbol{W}^{(l)}} = \delta^{(l)}(\boldsymbol{a}^{(l-1)})^{\mathsf{T}} \quad \in \mathbb{R}^{M_l \times M_{l-1}}.$$

$$\frac{\partial \mathcal{L}(\boldsymbol{y}, \hat{\boldsymbol{y}})}{\partial \boldsymbol{b}^{(l)}} = \delta^{(l)} \quad \in \mathbb{R}^{M_l}.$$

反向传播算法

算法 4.1 使用反向传播算法的随机梯度下降训练过程

输入: 训练集 $\mathcal{D} = \{(\boldsymbol{x}^{(n)}, y^{(n)})\}_{n=1}^{N}$, 验证集 $\mathcal{V}$, 学习率 α, 正则化系数 λ, 网络层数 L, 神经元数量 M_l, $1 \le l \le L$.

```
随机初始化 W, b;
repeat
    对训练集 D 中的样本随机重排序;
    for n = 1 ··· N do
        从训练集 D 中选取样本 (x^(n), y^(n));
        前馈计算每一层的净输入 z^(l) 和激活值 a^(l), 直到最后一层;
        反向传播计算每一层的误差 δ^(l);                    // 公式 (4.63)
        // 计算每一层参数的导数
        ∀l,  ∂L(y^(n), ŷ^(n)) / ∂W^(l) = δ^(l) (a^(l-1))^T;   // 公式 (4.68)
        ∀l,  ∂L(y^(n), ŷ^(n)) / ∂b^(l) = δ^(l);               // 公式 (4.69)
        // 更新参数
        W^(l) ← W^(l) − α(δ^(l) (a^(l-1))^T + λ W^(l));
        b^(l) ← b^(l) − α δ^(l);
    end
until 神经网络模型在验证集 V 上的错误率不再下降;
```

输出: $\boldsymbol{W}, \boldsymbol{b}$

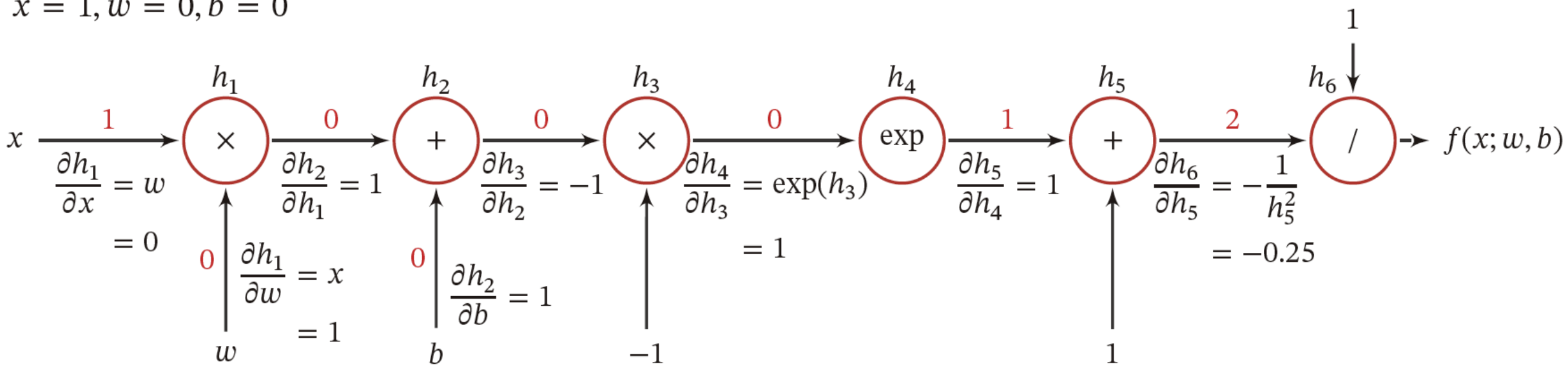
思考: 前馈计算中需要储存哪些值?

计算图与自动微分

自动微分是利用链式法则来自动计算一个复合函数的梯度

$$f(x;w,b)=\frac{1}{\exp\big(-(wx+b)\big)+1},$$

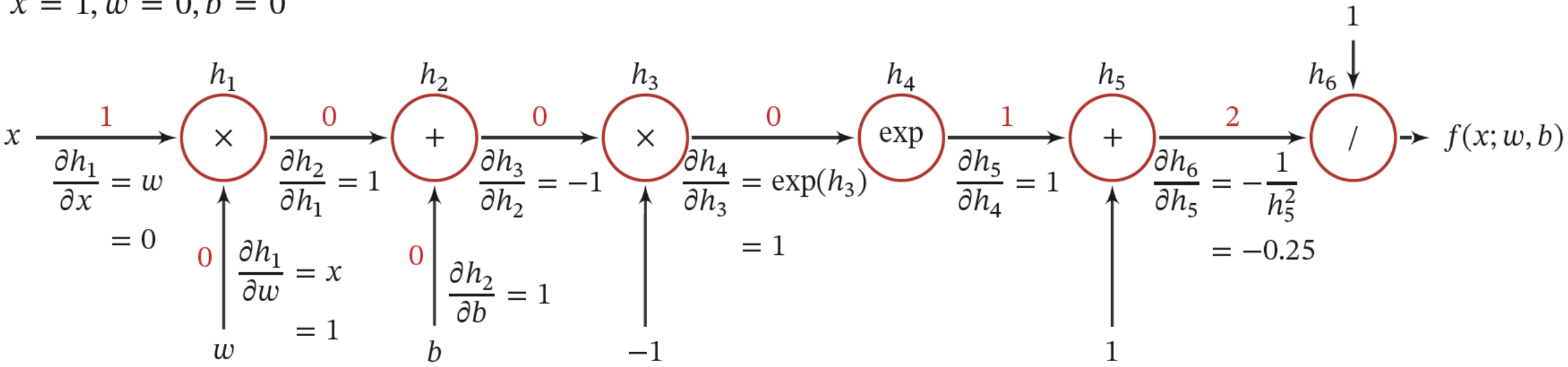
$x=1, w=0, b=0$



计算图

计算图与自动微分

$x = 1, w = 0, b = 0$

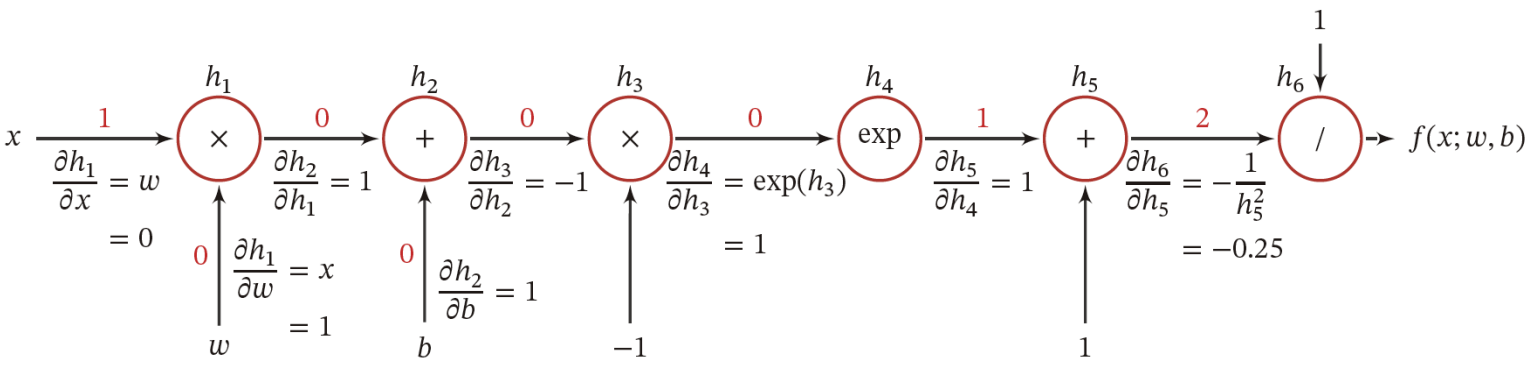


$$\frac{\partial f(x;w,b)}{\partial w}\Big|_{x=1,w=0,b=0} = \frac{\partial f(x;w,b)}{\partial h_6}\frac{\partial h_6}{\partial h_5}\frac{\partial h_5}{\partial h_4}\frac{\partial h_4}{\partial h_3}\frac{\partial h_3}{\partial h_2}\frac{\partial h_2}{\partial h_1}\frac{\partial h_1}{\partial w}$$

$$= 1 \times -0.25 \times 1 \times 1 \times -1 \times 1 \times 1$$

$$= 0.25.$$

计算图与自动微分



前向模式 按计算图中计算方向的相同方向来递归地计算梯度

反向模式 按计算图中计算方向的相反方向来递归地计算梯度

$$\frac{\partial f(x;w,b)}{\partial w} = \frac{\partial f(x;w,b)}{\partial h_6}\frac{\partial h_6}{\partial h_5}\frac{\partial h_5}{\partial h_4}\frac{\partial h_4}{\partial h_3}\frac{\partial h_3}{\partial h_2}\frac{\partial h_2}{\partial h_1}\frac{\partial h_1}{\partial w}$$

反向模式和反向传播的计算梯度的方式相同

若有多条路径, 则对多条路径的导数相加, 得到最终梯度

PyTorch的例子

```python
import torch

# 定义自变量 a, b, c
a = torch.tensor(2.0, requires_grad=True)
b = torch.tensor(3.0, requires_grad=True)
c = torch.tensor(4.0, requires_grad=True)

# 定义中间变量 x, y
x = a * b
y = b + c

# 定义最终的因变量 z
z = x ** 2 + y ** 2

# 计算 z 关于 a 的梯度
z.backward()

# 输出梯度
print(f"z关于a的梯度: {a.grad.item()}")
```

✓ 3.3s

```
z关于a的梯度: 36.0
```

反向传播算法 (自动微分的反向模式)

前馈神经网络的训练过程可以分为以下三步

- 前向计算每一层的状态和激活值，直到最后一层
- 反向计算每一层的参数的偏导数
- 更新参数

静态计算图和动态计算图

静态计算图是在编译时构建计算图, 计算图构建好之后在程序运行时不能改变

Tensorflow <2.0

动态计算图是在程序运行时动态构建

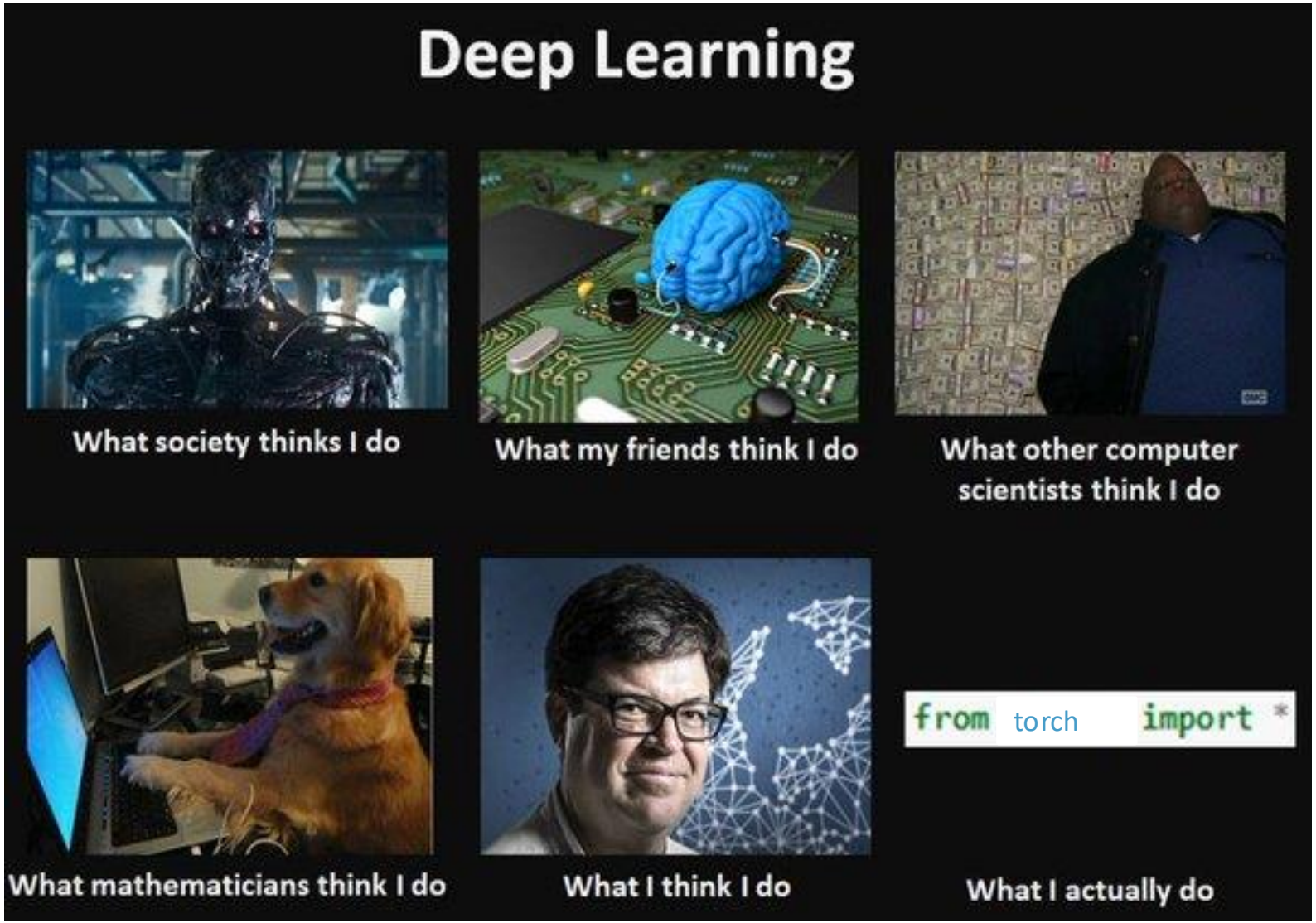
PyTorch

各有优缺点

静态计算图在构建时可以进行优化，并行能力强，但灵活性比较差低。

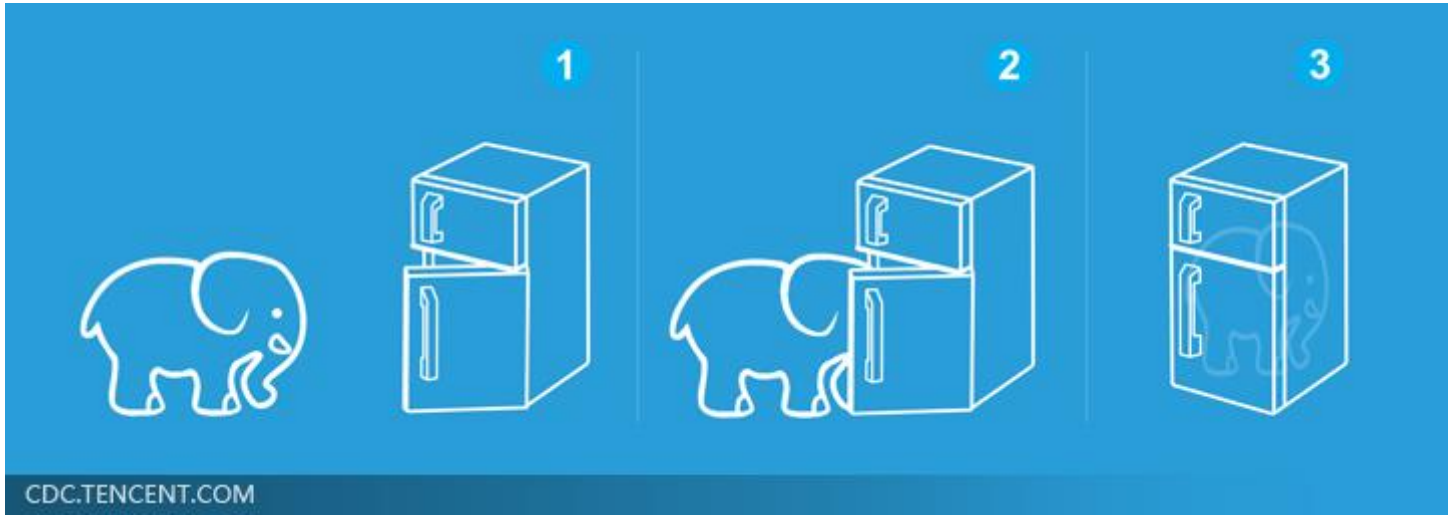
动态计算图则不容易优化，当不同输入的网络结构不一致时，难以并行计算，但是灵活性比较高。

如何实现


Deep Learning
What society thinks I do
What my friends think I do
What other computer scientists think I do
What mathematicians think I do
What I think I do
from torch import *
What I actually do

深度学习的三个步骤

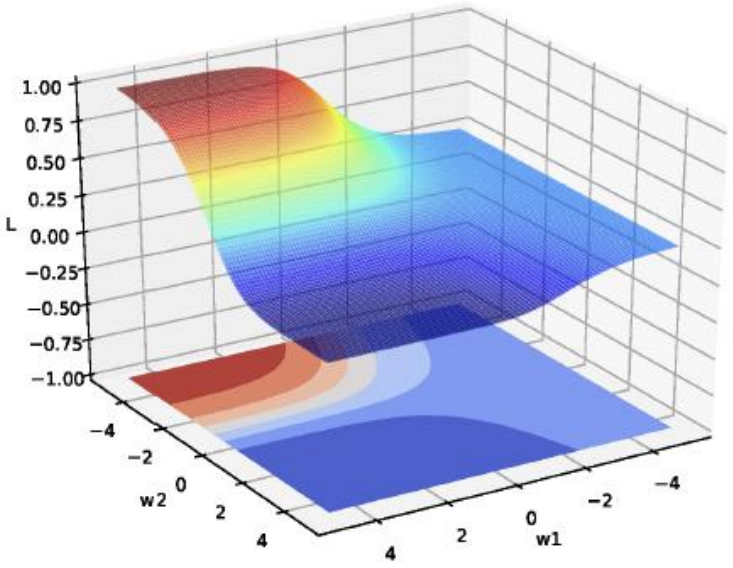
Deep Learning is so simple ……



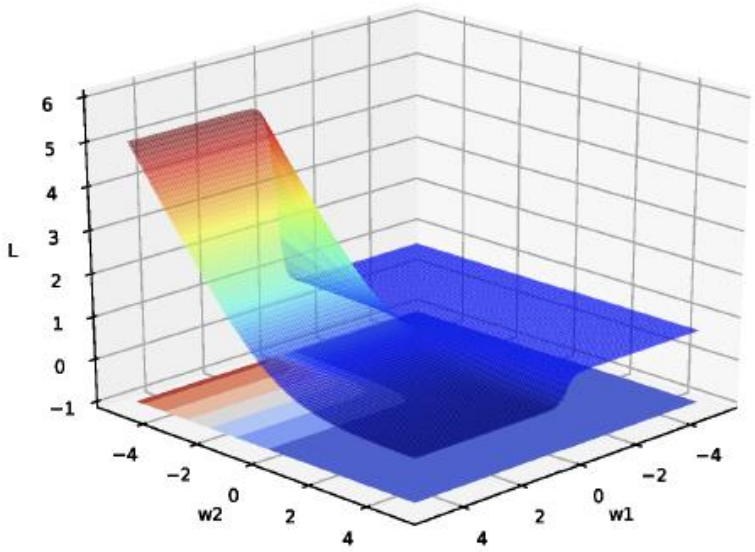
优化问题

非凸优化问题

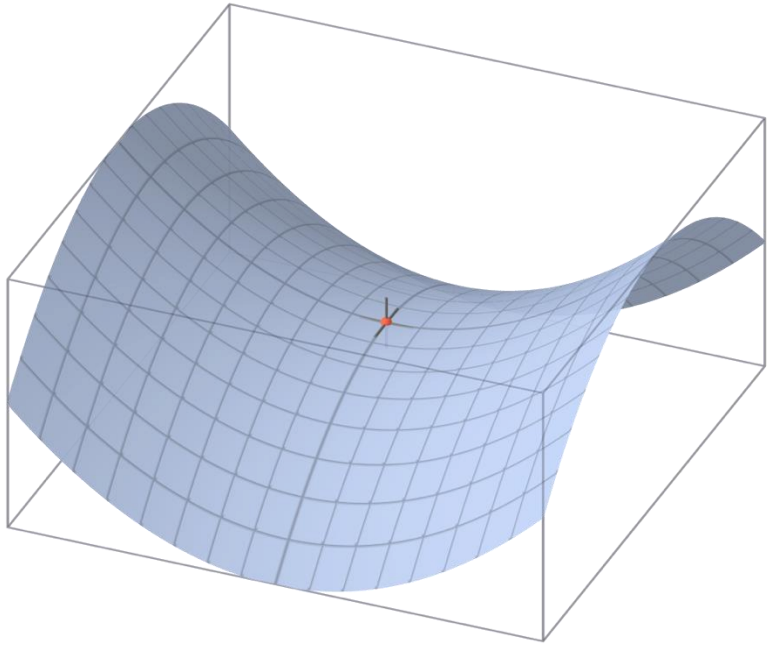
$$y = \sigma(w_2\sigma(w_1 x)),$$



(a) 平方误差损失



(b) 交叉熵损失

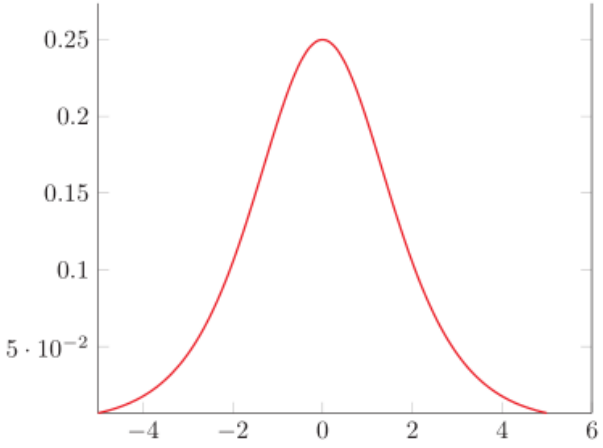


优化问题

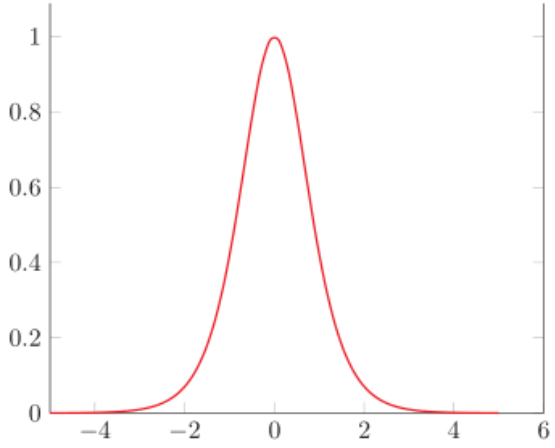
梯度消失问题

$$y = f^5(f^4(f^3(f^2(f^1(x)))))$$

$$\frac{\partial y}{\partial x} = \frac{\partial f^1}{\partial \mathrm{x}} \frac{\partial f^2}{\partial f^1} \frac{\partial f^3}{\partial f^2} \frac{\partial f^4}{\partial f^3} \frac{\partial f^5}{\partial f^4}$$



(a) logistic 函数的导数



(b) tanh 函数的导数

优化问题

难点

非凸优化问题：即存在局部最优而非全局最优解，影响迭代

梯度消失问题，下层参数比较难调

参数过多，影响训练

参数解释起来比较困难

需求

计算资源要大

数据要多

算法效率要好：即收敛快

谢谢！